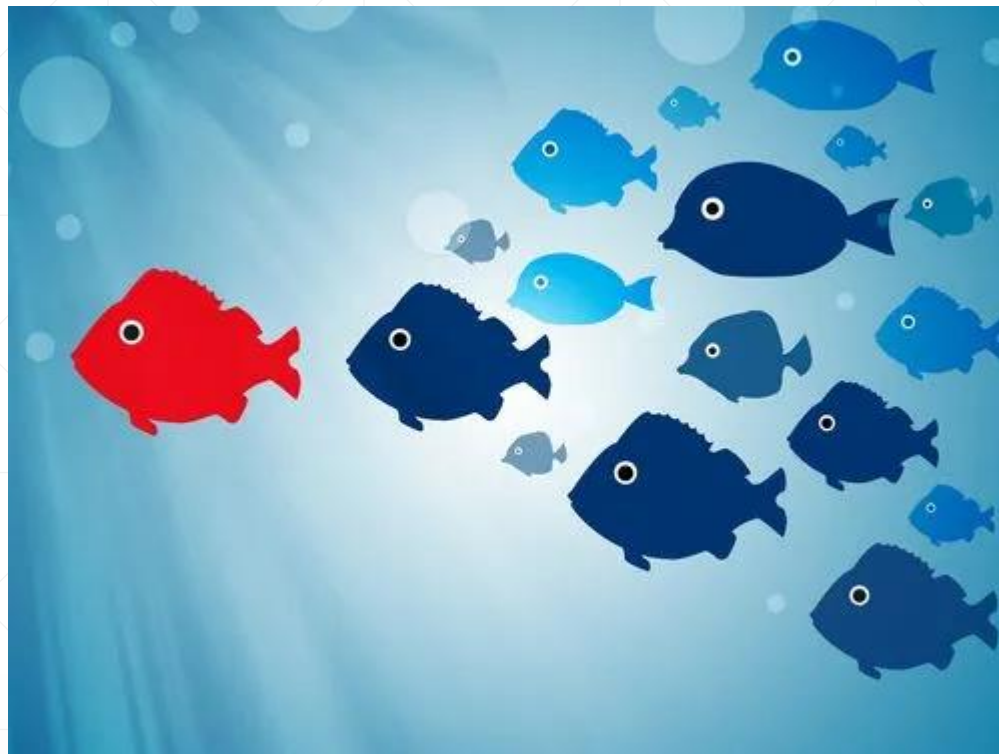




同步代码块

北京理工大学计算机学院
金旭亮

概述



通过给方法添加synchronized关键字，可以将一个普通的方法转换为同步方法。



同步方法能实现方法代码的线程互斥调用，但这种“锁定”的粒度比较大，如果方法体中的代码很多，则所有这些方法都只能被顺次独占式地调用和运行，可能会对程序性能有不好的影响。



事实上，通常只有一部分的代码是真需要同步互斥调用的，其它那些“彼此不相关”的代码，是完全可以并发运行的，没必要仅仅因为它们位于“同一个方法”内部，就必须一并“锁定”，“排队顺序运行”。

编写线程安全的代码—同步代码块



使用 **同步代码块**，可以仅“锁定”特定的代码：

```
...  
synchronized(lockObject) {  
    //需要同步的代码  
}  
...
```

上述代码中的lockObject代表**同步对象**，通常我们会将要访问此同步对象的代码放入到此同步块中（这并非是必须遵守的规则，完全可以放置任意的代码）。JVM会保证只有一个线程执行代码块中的这些代码。

同步方法可看成是同步代码块的特殊情况



同步方法（即在方法声明中使用了`synchronized`关键字），可以看成是锁定了整个实例对象和所有方法代码的同步代码块。

```
void synchronized func(){  
    ...  
}
```



```
void func(){  
    synchronized(this){  
        ...  
    }  
}
```

临界区 (critical section)



同步代码块，定义了一个“临界区”。



临界区是一个用于访问共享资源的代码块，并且在同一时刻只能由一个线程执行。



临界区可以使用`synchronized(lockObject)`的方式进行定义，也可以使用其他多线程组件，比如`ReentrantLock`定义，这些组件，都实现了`Lock`接口。

```
void func(){  
    synchronized(this){  
        ...  
    }  
}
```

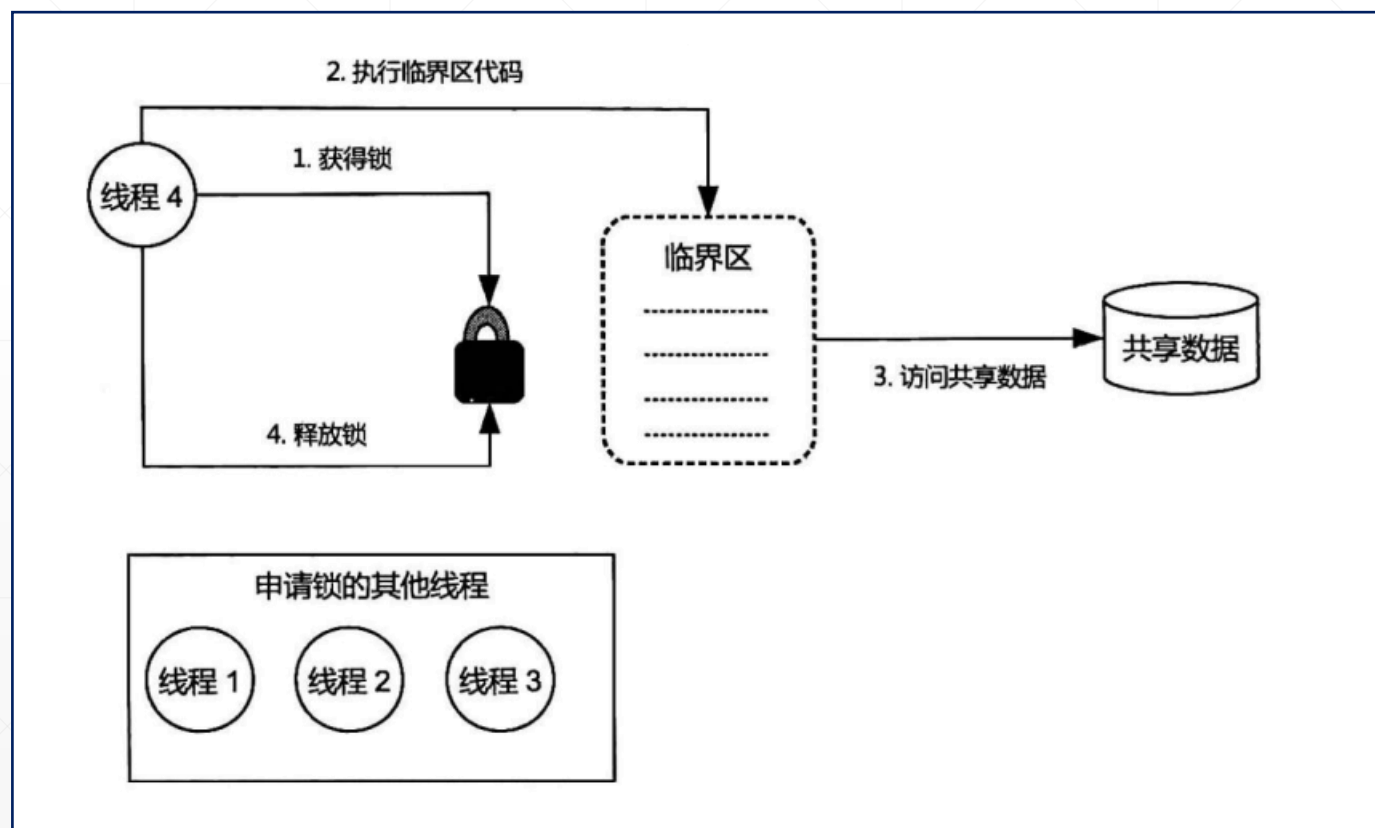
临界区



每一个使用`synchronized`关键字声明的方法（或代码块）都是一个临界区，Java在同一时刻只允许一个线程执行临界区内的代码。

“互斥锁”与“临界区”

“**临界区 (Critical Section)**”其实就是锁的持有线程在其获得锁之后和释放锁之前这段时间内所执行的代码。



了解技术内幕



编译器在编译“同步代码块”时，会在临界区相应字节码指令块前后添加`monitorenter`和`monitorexit`指令，以实现多线程互斥访问此代码块。

判断当前线程是否拥有了锁.....



```
public class HoldsLockTest {  
    public static void main(String[] args) {  
        HoldsLockTest obj = new HoldsLockTest();  
        obj.testObjectLock();  
    }  
  
    public void testObjectLock() {  
        boolean hasLock = false;  
        hasLock = Thread.holdsLock(obj: this);  
        System.out.println("当前线程执行有锁? " + hasLock);  
        synchronized (this) {  
            hasLock = Thread.holdsLock(obj: this);  
            System.out.println("当前线程执行有锁? " + hasLock);  
        }  
        hasLock = Thread.holdsLock(obj: this);  
        System.out.println("当前线程执行有锁? " + hasLock);  
    }  
}
```

每个同步代码块都关联着一个“锁”，Thread类定义了一个holdsLock()方法，用于判断当前正在执行的代码没有关联一个锁。

Run: HoldsLockTest x

"C:\Program Files\Java\jdk-17\bin\java.exe"

当前线程执行有锁? false

当前线程执行有锁? true

当前线程执行有锁? false

Process finished with exit code 0

思考与巩固:



```
public class WhyThreadNotRun {
    static void print(String info) throws InterruptedException {
        synchronized (info) {
            while (true) {
                System.out.println(Thread.currentThread().getName()
                    + " is working...");
                Thread.sleep(1000);
            }
        }
    }
}
```

上述示例代码运行时，会发现创建的两个线程，永远都只有一个线程在运行，而另一个线程陷入了“永久等待”的情形，为什么？解释原因！

```
public static void main(String[] args) {
    var threadA = new Thread(() -> {
        try {
            print("Hello");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }, name: "ThreadA");
    var threadB = new Thread(() -> {
        try {
            print("Hello");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }, name: "ThreadB");
    threadA.start();
    threadB.start();
}
```

WhyThreadNotRun.java



避免锁对程序性能的不利影响

锁定实例对象导致串行访问

```
class MyClass {  
    1 usage  
    public synchronized void f() throws InterruptedException {  
        String threadName = Thread.currentThread().getName();  
        System.out.println(threadName + "运行f()");  
        for (int i = 0; i < 5; i++) {  
            Thread.sleep( millis: 500);  
            System.out.println(threadName + ":f():" + i);  
        }  
    }  
}  
  
1 usage  
public synchronized void g() throws InterruptedException {  
    String threadName = Thread.currentThread().getName();  
    System.out.println(threadName + "运行g()");  
    for (int i = 0; i < 10; i++) {  
        Thread.sleep( millis: 200);  
        System.out.println(threadName + ":g():" + i);  
    }  
}
```

MyClass类中定义了两个同步方法，注意，这两个方法其实没有任何交集，理论上应该是可以并行运行的。

下面让我们写一个示例，启动两个线程，一个调用f()方法，另一个调用g()方法，看看这两个方法的执行情况。

多线程并发访问同一实例的不同同步方法

```
VisitMultiSyncMethodDemo.java x
1 public class VisitMultiSyncMethodDemo {
2     public static void main(String[] args) {
3         var obj = new MyClass();
4         Thread thread1 = new Thread(() -> {
5             try {
6                 obj.f();
7             } catch (InterruptedException e) {
8                 throw new RuntimeException(e);
9             }
10        });
11        Thread thread2 = new Thread(() -> {
12            try {
13                obj.g();
14            } catch (InterruptedException e) {
15                throw new RuntimeException(e);
16            }
17        });
18        thread1.start();
19        thread2.start();
20    }
21 }
```

```
Run: VisitMultiSyncMethodDemo x
"C:\Program Files\Java\jdk-17\bin\
Thread-0运行f()
Thread-0:f():0
Thread-0:f():1
Thread-0:f():2
Thread-0:f():3
Thread-0:f():4
Thread-1运行g()
Thread-1:g():0
Thread-1:g():1
Thread-1:g():2
Thread-1:g():3
Thread-1:g():4
Thread-1:g():5
Thread-1:g():6
Thread-1:g():7
Thread-1:g():8
Thread-1:g():9

Process finished with exit code 0
```

从结果中可以看到，两个线程是顺序执行的。

多线程访问同一个对象的同步方法，是互斥访问的

一种改进思路



如果同一个类中的两个方法是独立的（即没有访问相同的共享变量），那么，使用不同的锁，其实它们是可以被并行执行的，可以在内部放置一个专用于同步的对象，在同步方法内部锁定它：

```
class SynchronizedClass {  
  
    //专用的同步对象  
    private Object myLock = new Object();  
  
    public void printHello() {  
        synchronized (myLock) {  
            //同步代码块代码  
        }  
    }  
}
```

为不同的同步方法指定不同的
锁定对象，就能实现并行调用。

请动手一试！

参考示例: SynchronizedMethodTest.java

应用实例

JDK中有大量的代码采用了“同步代码块”这一特性，比如，天天见到的`System.out.println()`方法，它的一个重载形式的源码如下：

```
public void println( @Nullable String x) {  
    if (getClass() == PrintStream.class) {  
        writeln(String.valueOf(x));  
    } else {  
        synchronized (this) {  
            print(x);  
            newLine();  
        }  
    }  
}
```

同步代码块！

小结



同步方法是一种粗粒度的并发控制手段，某一时刻只能有一个线程执行该方法。同步代码块则是一种细粒度的并发控制，只会将块中的代码同步，位于方法内同步代码块之外的代码是可以被多个线程同时访问到的。



不恰当的锁定，可能会带来严重的性能问题。



推荐尽量采用同步代码块的方式，以减少不必要的对象锁定，尽量减少同步代码块的运行时间，提升程序的性能。

巩固练习



去医院看病，通常需要遵循特定的流程：
挂号-->分诊-->就诊-->交费-->取药

可以编写一个Hosptial类代表医院，上述流程中的每个步骤设置为此类的同步方法。

用线程代表一个病人，假设每个流程步骤中，一次只能为一个病人服务。

请写一个多线程程序，模拟多个病人到医院就医的过程。